

Sprint 1 Retrospective Report and Sprint 2 Planning

Project: LoraFilm - Online Movie Ticket Booking System

Team: Group 3

Sprint: Sprint 1 - Authentication, Architecture & UI Foundation

Report Type: Sprint Retrospective, Sprint Review and Sprint 2 Planning

Sprint Focus: Authentication, User Profile, Frontend Integration, Kafka Event Flow, Architecture & API Contracts

Updated date: 18/06/2026

1. Tổng Quan Sprint 1

Sprint 1 là sprint xây dựng nền tảng kỹ thuật đầu tiên cho hệ thống **LoraFilm**. Nếu Sprint 0 tập trung vào setup project, GitLab workflow và tài liệu kiến trúc ban đầu, thì Sprint 1 tập trung vào việc hiện thực hóa các thành phần nền tảng có thể chạy được trong hệ thống.

Trong Sprint 1, team đã triển khai được luồng **Authentication & User Management** gồm đăng ký tài khoản, xác thực OTP, đăng nhập, refresh token, tạo hồ sơ người dùng và lấy thông tin hồ sơ người dùng. Bên cạnh đó, Frontend đã được chuyển từ giai đoạn mock/prototype sang repo thật, sử dụng API thật thông qua API Gateway.

Một điểm quan trọng trong Sprint 1 là team đã hoàn thành flow giao tiếp bất đồng bộ giữa **auth-service** và **user-service** bằng Kafka thông qua event **ACCOUNT_CREATED**. Sau khi người dùng đăng ký thành công, Auth Service publish event lên Kafka, User Service consume event và tự động tạo user profile.

Ngoài phần code, team cũng hoàn thiện nhiều tài liệu kỹ thuật quan trọng như API Specification, Event Contract, Frontend Integration Specification và tài liệu kiểm thử tích hợp Auth flow.

2. Sprint Goal

Mục tiêu chính của Sprint 1:

- Xây dựng luồng đăng ký, xác thực OTP và đăng nhập cho người dùng.
 - Tích hợp JWT Access Token và Refresh Token.
 - Xây dựng User Profile Management cơ bản.
 - Kết nối Frontend Auth UI với Backend thông qua API Gateway.
 - Tích hợp CCCD Check API vào Register flow.
 - Hoàn thiện Kafka event **ACCOUNT_CREATED** giữa Auth Service và User Service.
 - Chuẩn hóa tài liệu API Contract, Event Contract và Frontend Integration.
 - Kiểm thử luồng Authentication end-to-end từ Frontend đến Backend.
-

3. Sprint 1 Scope

Sprint 1 bao gồm các nhóm công việc chính:

3.1. Authentication & Authorization

- Register API.
- Verify OTP API.
- Login API.
- Refresh Token API.
- Hash password.
- JWT Access Token.
- Refresh Token lưu database.
- Account trạng thái chưa xác thực OTP.
- Role mặc định **CUSTOMER**.

3.2. User Profile Management

- Internal Create User Profile API.
- Get User Profile API.
- Lưu thông tin profile ở **user-service**.
- Không trả full CCCD ở API public.
- Chỉ trả **cccdMasked** cho Frontend.
- Kafka Consumer tạo profile từ event **ACCOUNT_CREATED**.

3.3. Kafka Event Integration

- Auth Service đóng vai trò Kafka Producer.
- User Service đóng vai trò Kafka Consumer.
- Topic: **auth.account.created.v1**.
- Message key: **accountId**.
- Event type: **ACCOUNT_CREATED**.
- Consumer group: **user-service-account-created-consumer**.
- Idempotency rule: không tạo trùng profile nếu event bị gửi lặp.

3.4. Frontend Integration

- Kéo các màn hình từ UI prototype về repo Frontend thật.
- Frontend thật không dùng mock business data.
- Các màn chưa có API thật giữ UI shell hoặc empty state.
- Home page.
- Register page.
- Login page.
- Verify OTP page.
- Customer Profile page.
- Header authentication state.
- Service layer: **authService.js**, **userService.js**, **cccdService.js**.
- Auth storage: **authStorage.js**.
- Chỉ gọi API qua API Gateway.

3.5. API Gateway

- Route **/api/auth/**** đến Auth Service.

- Route `/api/users/**` đến User Service.
- Frontend không gọi trực tiếp port service nội bộ.
- Phát hiện và ghi nhận issue bảo mật liên quan đến route `/internal/users/**`.

3.6. Documentation & Testing

- Auth Service API Specification.
- User Service API Specification.
- Kafka `ACCOUNT_CREATED` Event Contract.
- Frontend Auth & Real API Integration Specification.
- Auth Flow E2E Test Cases.
- Ghi nhận discrepancy giữa contract và implementation.

4. Sprint 1 Deliverables

4.1. Auth Service Deliverables

Auth Service đã hoàn thành:

- `POST /api/auth/register`
- `POST /api/auth/verify`
- `POST /api/auth/login`
- `POST /api/auth/refresh-token`
- OTP verification flow.
- JWT Access Token.
- Refresh Token.
- Kafka Producer publish `ACCOUNT_CREATED`.
- Account mặc định role `CUSTOMER`.
- Account mới ở trạng thái chưa xác thực OTP.
- Password được hash trước khi lưu.

4.2. User Service Deliverables

User Service đã hoàn thành:

- `POST /internal/users`
- `GET /api/users/{accountId}`
- Kafka Consumer listen topic `auth.account.created.v1`
- Tự động tạo user profile từ Kafka event.
- Idempotency check theo `accountId`.
- Public API chỉ trả `cccdMasked`, không trả full CCCD.
- Response field `verifiedPhone`.

4.3. Frontend Deliverables

Frontend đã hoàn thành:

- Migrate nhiều màn hình từ FE prototype vào repo thật.
- Register form kết nối API thật.

- CCCD Check API integration.
- Birthday mismatch validation.
- Verify OTP page.
- Login form kết nối API thật.
- Token storage validation.
- Customer Profile service.
- Header auth state.
- Empty state cho các màn chưa có API thật.
- Không dùng mock business data trong repo thật.

4.4. Kafka Deliverables

Kafka flow đã hoàn thành và test pass:

```
Customer Register
→ API Gateway
→ Auth Service
→ Save account
→ Publish ACCOUNT_CREATED
→ Kafka topic auth.account.created.v1
→ User Service Consumer
→ Create user profile
→ GET /api/users/{accountId}
```

4.5. Documentation Deliverables

Các tài liệu chính trong Sprint 1:

```
docs/api/auth-service-api.md
docs/api/user-service-api.md
docs/events/account-created-event.md
docs/frontend/auth-integration.md
docs/test/auth-flow-test-cases.md
```

5. Sprint 1 Test Summary

Sprint 1 đã có tài liệu kiểm thử tích hợp cho luồng Auth/Register/Login.

Các nhóm test chính:

Nhóm test	Kết quả
Register success E2E	ĐẠT
CCCD Check API	ĐẠT
Birthday mismatch validation	ĐẠT

Nhóm test	Kết quả
Duplicate email / phone / CCCD	ĐẠT
Login success	ĐẠT
Invalid login credentials	ĐẠT
Account not verified handling	CHƯA ĐẠT
Token storage validation	ĐẠT
Kafka account created flow	ĐẠT
User profile created from Kafka event	ĐẠT

6. Discrepancy / Issue Tracking

Trong quá trình kiểm thử và đối chiếu contract, team ghi nhận một số điểm lệch cần xử lý hoặc theo dõi.

6.1. Login Response Thiếu Field `token`

Auth contract có mô tả field `token` để tương thích ngược, nhưng implementation thực tế có thời điểm chỉ trả `accessToken`, `refreshToken`, `expiresIn`, `tokenType`, `email`, `role`.

Ảnh hưởng:

- Frontend đã fallback bằng `data.accessToken || data.token`.
- Không làm vỡ flow login.
- Tuy nhiên Backend nên đồng bộ lại contract hoặc cập nhật tài liệu nếu quyết định bỏ `token`.

6.2. Gateway Expose `/internal/users/**`

Theo contract, `/internal/users` là Internal API và không được expose cho Frontend hoặc external client.

Thực tế kiểm thử phát hiện Gateway có route public cho `/internal/users/**`.

Ảnh hưởng:

- Đây là rủi ro bảo mật cao.
- Cần remove route hoặc chặn bằng internal token/security filter.
- Nên tạo issue bug/security riêng.

6.3. Account Not Verified Handling Chưa Đạt

Khi tài khoản chưa xác thực OTP đăng nhập lại, kỳ vọng là trả `AUTH_ACCOUNT_NOT_VERIFIED` và Frontend điều hướng sang `/verify-otp`.

Kết quả thực tế:

- UI hiển thị lỗi chung "Email hoặc mật khẩu không chính xác".
- Chưa điều hướng đúng về Verify OTP.

Hướng xử lý:

- Backend cần trả đúng `AUTH_ACCOUNT_NOT_VERIFIED`.
- Nếu có thể, response nên kèm `accountId` để Frontend set `pendingAccountId`.
- Frontend update error mapping sau khi BE response chuẩn.

6.4. Java Field Mismatch `isVerifiedPhone` / `verifiedPhone`

Response JSON hiện trả `verifiedPhone` đúng nhờ getter thủ công, nhưng code nội bộ vẫn có thể gây nhầm lẫn bảo trì.

Hướng xử lý:

- Đồng bộ tên field trong DTO để dễ maintain.

7. Sprint 1 Result

Hạng mục	Kết quả
Authentication APIs	Completed
User Profile APIs	Completed
Frontend Auth Integration	Completed
Kafka <code>ACCOUNT_CREATED</code> Flow	Completed and Passed
API Gateway Auth/User Routing	Completed, cần fix route internal
Documentation	Completed / Updated
E2E Auth Test Cases	Completed
Major blocker	Không còn blocker chính
Security issue	Gateway expose <code>/internal/users/**</code> cần xử lý

Tổng kết: Sprint 1 đạt mục tiêu chính. Một số discrepancy được ghi nhận rõ để đưa vào Sprint 2 hoặc xử lý bug riêng.

8. What Went Well

Trong Sprint 1, team làm tốt các điểm sau:

- Team đã hoàn thành được một luồng nghiệp vụ nền tảng có giá trị cao: Register → OTP → Login → Profile.
- Auth Service và User Service đã có sự phân tách trách nhiệm rõ hơn.
- Kafka `ACCOUNT_CREATED` flow đã chạy được và test pass.
- Frontend đã không còn phụ thuộc mock business data cho Auth flow.
- API Gateway đã trở thành điểm vào chính cho Frontend.
- API Contract giúp BE và FE phối hợp tốt hơn.
- Event Contract giúp Vinh và Thành thống nhất topic/payload/consumer behavior.
- Test case giúp team phát hiện được các lỗi lệch contract và rủi ro bảo mật.
- Team đã có ý thức cập nhật tài liệu để mentor review rõ hơn.

9. What Needs Improvement

Một số điểm cần cải thiện trong Sprint 2:

- Cần cập nhật tài liệu song song với code ngay trong cùng MR.
 - Cần review Gateway route kỹ hơn để tránh expose internal API.
 - Cần chuẩn hóa response error giữa các service.
 - Cần trả `accountId` rõ ràng trong Login response hoặc thống nhất JWT claim.
 - Cần viết test case sớm hơn, không đợi gần cuối sprint.
 - Cần tách bug/security issue ra khỏi feature issue để dễ tracking.
 - Cần có checklist demo mentor trước buổi review.
 - Cần thống nhất format tài liệu API/Event/Frontend giữa các member.
-

10. Lessons Learned

Các bài học chính sau Sprint 1:

- Mock data giúp build UI nhanh, nhưng repo thật cần có nguyên tắc rõ: API thật hoặc empty state.
 - API Gateway không chỉ là route, mà còn là điểm kiểm soát bảo mật.
 - Internal API cần được bảo vệ hoặc không expose qua Gateway.
 - Kafka event cần contract rõ ràng trước khi code producer/consumer.
 - Consumer phải xử lý idempotency vì event có thể bị gửi lặp.
 - Với dữ liệu nhạy cảm như CCCD, public API chỉ nên trả masked value.
 - Frontend nên lưu token tập trung bằng utility thay vì thao tác localStorage rải rác.
 - Testing E2E giúp phát hiện lỗi contract mà unit test từng service có thể bỏ sót.
-

11. Challenges / Blockers and Action Plan

Trong Sprint 1, team gặp một số khó khăn trong quá trình triển khai Authentication, User Profile, Frontend Integration, API Gateway và Kafka event flow. Các vấn đề dưới đây không chỉ được ghi nhận lại như “khó khăn đã xảy ra”, mà còn được phân tích theo hướng:

Vấn đề → Nguyên nhân → Tác động → Tình trạng hiện tại → Hành động đã thực hiện → Kế hoạch phòng ngừa

Mục tiêu của phần này là giúp team rút kinh nghiệm rõ ràng, tránh việc thành viên mới hoặc thành viên khác lặp lại cùng một lỗi trong các sprint sau.

11.1. Backend - Frontend API Contract từng bị lệch trong lúc implement

Vấn đề đã xảy ra: Trong quá trình Sprint 1, Auth Service, User Service và Frontend được phát triển song song nên có thời điểm API response thực tế chưa khớp hoàn toàn với API Contract. Một số điểm từng bị lệch gồm Login response, field `token` / `accessToken`, `accountId` phục vụ Profile flow và một số `errorCode` phục vụ xử lý lỗi ở Frontend.

Nguyên nhân:

- Backend và Frontend được implement song song.
- API Contract thay đổi trong lúc code.
- Một số thay đổi được fix nhanh trong quá trình tích hợp nhưng chưa được tách thành issue riêng.
- Team chưa có rule bắt buộc mọi thay đổi contract phải được ghi rõ trong issue hoặc MR note.

Ảnh hưởng:

- Frontend từng phải thêm fallback tạm thời như `data.accessToken || data.token`.
- Profile flow có thể bị khó xử lý nếu Login response thiếu `accountId`.
- Mentor hoặc member mới có thể khó trace lại vì sao contract thay đổi.
- Nếu không ghi nhận lại, lỗi tương tự có thể lặp lại ở Sprint 2 khi nhiều service mới bắt đầu implement API.

Trạng thái hiện tại: Đã xử lý trong Sprint 1.

Service owner đã cập nhật lại các tài liệu liên quan:

```
docs/api/auth-service-api.md
docs/api/user-service-api.md
docs/frontend/auth-integration.md
```

Frontend cũng đã điều chỉnh service layer và auth storage để đọc response đúng theo contract hiện tại.

Action đã thực hiện:

- Cập nhật lại Auth Service API Contract.
- Cập nhật lại User Service API Contract.
- Cập nhật lại Frontend Auth Integration document.
- Frontend bổ sung xử lý response phù hợp với contract hiện tại.
- Ghi nhận discrepancy trong Sprint 1 Retrospective Report.

Bài học rút ra:

API Contract phải được xem như giao ước chính thức giữa Backend và Frontend. Khi contract thay đổi, tài liệu và code phải được cập nhật cùng lúc, đồng thời phải có issue hoặc MR note để trace lại lịch sử thay đổi.

Action phòng tránh cho Sprint 2:

Action	Owner	Áp dụng từ
API mới phải có API Contract trước hoặc song song lúc implement	Service owner	Sprint 2
Nếu thay đổi request/response/errorCode thì phải cập nhật tài liệu trong cùng MR	Service owner	Sprint 2
Không fix âm thầm contract nếu thay đổi ảnh hưởng FE/BE khác	Cả team	Sprint 2

Action	Owner	Áp dụng từ
MR phải có checklist "API Contract updated if needed"	Reviewer / Leader	Sprint 2
Frontend review contract trước khi nối API thật	Nhân	Sprint 2

Prevention rule:

Không merge API implementation nếu API Contract chưa được cập nhật hoặc reviewer chưa xác nhận contract khớp với implementation.

11.2. API Gateway expose nhầm Internal API

Vấn đề đã xảy ra: Endpoint `/internal/users/**` là Internal API, chỉ dùng cho backend/internal service communication. Tuy nhiên trong Sprint 1, route này vẫn bị expose qua API Gateway.

Nguyên nhân:

- Khi cấu hình Gateway, team tập trung vào việc route chạy được trước.
- Chưa review đủ góc độ bảo mật cho internal route.
- Chưa có checklist phân biệt rõ Public API, Protected API và Internal API.
- Internal endpoint chưa có cơ chế bảo vệ bằng internal token hoặc network restriction.

Ảnh hưởng:

- Đây là rủi ro bảo mật cao.
- External client có thể gọi nhầm hoặc cố tình gọi `/internal/users`.
- Nếu không ghi nhận lại, Sprint sau khi thêm nhiều service có thể tiếp tục expose nhầm internal API khác.
- Mentor có thể đánh giá đây là lỗi kiến trúc/bảo mật nghiêm trọng nếu không có action xử lý rõ ràng.

Trạng thái hiện tại: Chưa nên xem là hoàn tất nếu route vẫn còn expose. Cần tạo bug/security issue riêng trong Sprint 2 hoặc xử lý ngay trước khi đóng Sprint 1 nếu còn thời gian.

Action cần thực hiện:

- Remove route `/internal/users/**` khỏi API Gateway nếu Frontend không cần gọi.
- Nếu vẫn cần route nội bộ trong môi trường dev, phải bảo vệ bằng internal token hoặc filter.
- Tạo issue bug/security riêng để tracking.
- Thêm checklist Gateway route review vào Sprint 2.

Action item cho Sprint 2:

Action	Owner đề xuất	Deadline
Tạo bug issue: Secure Internal Gateway Routes	Nhân / Khang	Đầu Sprint 2

Action	Owner đề xuất	Deadline
Remove hoặc protect <code>/internal/users/**</code>	Khang	Sprint 2
Review toàn bộ Gateway routes hiện có	Khang + Nhân	Sprint 2
Thêm rule: Internal API không expose qua Gateway	Cả team	Sprint 2

Prevention rule:

Mỗi route Gateway mới phải được phân loại rõ trước khi merge:

```
Public API    → External client có thể gọi
Protected API → Client gọi được nhưng cần authentication/authorization
Internal API  → Chỉ dùng nội bộ giữa service, không expose ra Gateway
```

Internal API mặc định không được expose qua API Gateway.

11.3. Kafka là công nghệ mới nhưng team đã hoàn thành flow chính

Vấn đề đã xảy ra: Kafka Producer/Consumer là phần mới với nhiều thành viên trong team. Ban đầu team cần thời gian để thống nhất topic, event payload, message key, consumer group và idempotency rule giữa Auth Service và User Service.

Nguyên nhân:

- Kafka là công nghệ mới với team.
- Producer và Consumer thuộc hai service khác nhau nên dễ lệch payload nếu không có contract.
- Team cần hiểu thêm về topic, message key, consumer group, retry và idempotency.
- Ban đầu team chưa có nhiều kinh nghiệm test flow bất đồng bộ end-to-end.

Ảnh hưởng:

- Việc tích hợp Auth Service và User Service mất nhiều thời gian hơn so với REST API thông thường.
- Nếu không có Event Contract, Producer và Consumer có thể implement lệch payload.
- Nếu không có idempotency, User Service có thể tạo trùng profile khi event bị gửi lại.
- Debug khó hơn vì phải kiểm tra nhiều điểm: Auth log, Kafka topic, User log và database.

Trạng thái hiện tại: Đã xử lý trong Sprint 1.

Team đã hoàn thành Kafka flow:

```
Auth Service
→ Publish ACCOUNT_CREATED
→ Kafka topic auth.account.created.v1
→ User Service Consumer
→ Create user profile
```

Event Contract đã được tạo tại:

```
docs/events/account-created-event.md
```

Flow Kafka **ACCOUNT_CREATED** đã test pass.

Action đã thực hiện:

- Tạo Event Contract cho **ACCOUNT_CREATED**.
- Chốt topic **auth.account.created.v1**.
- Chốt message key là **accountId**.
- Chốt consumer group **user-service-account-created-consumer**.
- User Service xử lý idempotency trước khi insert profile.
- Test flow Kafka bằng log và database.
- Ghi nhận kết quả pass trong report.

Bài học rút ra:

Với giao tiếp bất đồng bộ, Event Contract quan trọng tương đương API Contract. Producer và Consumer phải thống nhất tài liệu trước khi code.

Action phòng tránh cho Sprint 2:

Action	Owner	Áp dụng từ
Event mới phải có docs/events/<event-name>.md trước hoặc song song khi code	Service owner	Sprint 2
Event Contract phải ghi rõ topic, payload, message key, producer, consumer	Service owner	Sprint 2
Consumer phải có idempotency rule nếu event có thể bị gửi lặp	Consumer owner	Sprint 2
Kafka flow phải có test checklist riêng	BE Lead / Service owner	Sprint 2
Không merge Producer/Consumer nếu payload không khớp contract	Reviewer	Sprint 2

Prevention rule:

Mọi event mới phải có tài liệu tối thiểu:

```
Topic
Event type
Producer
Consumer
Payload
Message key
```

Retry rule
Idempotency rule

11.4. Frontend prototype đã được migrate sang repo thật và chuyển sang API thật

Vấn đề đã xảy ra: Frontend ban đầu được xây dựng từ UI prototype, nhiều màn hình dùng mock data để demo nhanh. Khi đưa vào repo thật, team phải chuyển sang nguyên tắc dùng API thật qua API Gateway và loại bỏ mock business data ở các flow đã có backend.

Nguyên nhân:

- Prototype ưu tiên tốc độ demo UI nên có mock data.
- Repo thật cần chuẩn hơn để mentor không hiểu nhầm mock là API thật.
- Nhiều màn được migrate cùng lúc nên cần phân biệt màn nào đã có API, màn nào chỉ là UI shell.
- Frontend cần refactor lại service layer thay vì gọi dữ liệu trực tiếp trong component.

Ảnh hưởng:

- Nếu không tách rõ mock và API thật, mentor có thể hiểu nhầm trạng thái hoàn thành của hệ thống.
- Frontend dễ gọi nhầm dữ liệu giả thay vì API thật.
- Các màn chưa có backend có thể gây hiểu nhầm nếu vẫn hiển thị dữ liệu mock.
- Khó kiểm thử E2E vì không rõ dữ liệu đến từ API hay mock.

Trạng thái hiện tại: Đã xử lý trong Sprint 1 cho Auth flow.

Frontend đã có tài liệu:

`docs/frontend/auth-integration.md`

Repo thật hiện theo nguyên tắc:

API thật nếu backend đã có
Empty state nếu backend chưa có
Không dùng mock business data trong repo thật

Các màn Auth như Register, Verify OTP, Login và Customer Profile đã được nối API thật qua API Gateway.

Action đã thực hiện:

- Migrate các màn từ FE prototype vào repo thật.
- Tạo service layer: `authService.js`, `userService.js`, `cccdService.js`.
- Tạo `authStorage.js` để quản lý token tập trung.
- Auth flow chuyển sang gọi API thật qua Gateway.
- Các màn chưa có API thật giữ UI shell hoặc empty state.
- Cập nhật `docs/frontend/auth-integration.md`.

Bài học rút ra:

Frontend prototype và frontend repo thật có mục tiêu khác nhau. Prototype dùng để mô phỏng UI, còn repo thật phải phản ánh đúng trạng thái tích hợp API.

Action phòng tránh cho Sprint 2:

Action	Owner	Áp dụng từ
Màn nào có API thì phải gọi API thật qua Gateway	Nhân	Sprint 2
Màn chưa có API thì dùng empty state, không dùng mock business data	Nhân	Sprint 2
Service layer phải tách khỏi component/page	Nhân	Sprint 2
Không hardcode service port nội bộ trong Frontend	Nhân	Sprint 2
Mỗi module FE mới cần ghi rõ trạng thái: API integrated / waiting for API	Nhân	Sprint 2

Prevention rule:

Frontend thật chỉ có hai trạng thái hợp lệ:

API thật đã nối
hoặc Empty state chờ API

Không dùng mock business data trong repo thật nếu không có ghi chú rõ ràng.

11.5. Login response và flow tài khoản chưa xác thực chưa ổn định

Vấn đề đã xảy ra: Kịch bản tài khoản chưa verify OTP đăng nhập lại chưa đạt hoàn toàn. Kỳ vọng là Backend trả `AUTH_ACCOUNT_NOT_VERIFIED` và Frontend điều hướng về `/verify-otp`, nhưng thực tế UI hiển thị lỗi chung "Email hoặc mật khẩu không chính xác".

Nguyên nhân:

- Backend chưa phân biệt rõ invalid credentials và account not verified.
- Response lỗi chưa đủ dữ liệu để Frontend set `pendingAccountId`.
- Error mapping phía Frontend phụ thuộc vào `errorCode`, nên nếu Backend trả sai `errorCode` thì Frontend không thể xử lý đúng.

Ảnh hưởng:

- User mới đăng ký nhưng chưa nhập OTP có trải nghiệm không rõ ràng.
- Frontend không thể tự động chuyển người dùng về Verify OTP.
- Test case `Account not verified handling` chưa đạt.
- Nếu không ghi rõ, Sprint sau có thể tiếp tục bỏ sót các error case tương tự.

Trạng thái hiện tại: Chưa đạt hoàn toàn theo test case Sprint 1. Cần tách bug issue để xử lý trong Sprint 2 hoặc xử lý trước khi đóng Sprint 1 nếu còn thời gian.

Action cần thực hiện:

- Backend cần trả đúng `AUTH_ACCOUNT_NOT_VERIFIED` khi tài khoản tồn tại nhưng chưa xác thực.
- Response nên kèm `accountId` nếu phù hợp để Frontend set `pendingAccountId`.
- Frontend update error mapping sau khi Backend response chuẩn.
- Tách issue bug riêng cho flow này.
- Retest lại test case `Account not verified handling`.

Action item cho Sprint 2:

Action	Owner đề xuất	Deadline
Tạo bug issue cho <code>AUTH_ACCOUNT_NOT_VERIFIED</code>	Nhân / Vinh	Đầu Sprint 2
Backend sửa <code>errorCode</code> và response	Vinh	Sprint 2
Frontend update redirect <code>/verify-otp</code>	Nhân	Sau khi BE fix
Retest test case Account Not Verified	Nhân	Sprint 2

Prevention rule:

Mỗi error case trong API Contract phải có đủ:

```
HTTP status
errorCode
message
data cần thiết cho Frontend xử lý
```

11.6. Tài liệu chưa được cập nhật đồng bộ với code ở mọi thời điểm

Vấn đề đã xảy ra: Một số tài liệu phải cập nhật lại sau khi code đã thay đổi, ví dụ Auth Service chuyển sang Kafka Producer, User Service có Kafka Consumer, Frontend chuyển từ mock/prototype sang API thật.

Nguyên nhân:

- Member thường ưu tiên code trước, tài liệu cập nhật sau.
- Chưa có checklist bắt buộc trong MR về việc cập nhật tài liệu.
- Một số thay đổi kiến trúc nhỏ không được ghi nhận ngay.
- Một số fix được làm nhanh nhưng chưa có issue/MR note rõ ràng.

Ảnh hưởng:

- Mentor review có thể thấy tài liệu và code lệch nhau.
- Member mới vào dễ đọc nhầm tài liệu cũ và làm sai tiếp.
- Các lỗi cũ có thể bị lặp lại vì không có documented action.
- Team khó trace lại lý do vì sao một field/flow bị thay đổi.

Trạng thái hiện tại: Đã cải thiện trong Sprint 1 vì các tài liệu chính đã được bổ sung/cập nhật. Tuy nhiên, quy trình cần được chuẩn hóa lại trong Sprint 2 để tránh fix âm thầm không có trace.

Action đã thực hiện:

- Cập nhật Auth Service API document.
- Cập nhật User Service API document.
- Cập nhật Event Contract cho Kafka `ACCOUNT_CREATED`.
- Cập nhật Frontend Auth Integration document.
- Ghi nhận discrepancy trong Sprint 1 Retrospective Report.

Action phòng tránh cho Sprint 2:

Action	Owner	Áp dụng từ
Thêm checklist documentation vào MR template	Leader / Nhân	Sprint 2
API change phải update <code>docs/api</code>	Service owner	Mọi MR liên quan API
Event change phải update <code>docs/events</code>	Service owner	Mọi MR liên quan Kafka
Frontend flow change phải update <code>docs/frontend</code>	FE owner	Mọi MR liên quan FE
Bug/contract/security fix phải có issue hoặc MR note	Cả team	Sprint 2

Prevention rule:

Không coi task là Done nếu code đã merge nhưng tài liệu liên quan chưa cập nhật.

11.7. Một số vấn đề đã fix nhưng chưa có issue riêng

Vấn đề đã xảy ra: Trong Sprint 1, một số vấn đề được team phát hiện và xử lý nhanh trong quá trình implement nhưng chưa được tách thành issue riêng.

Ví dụ:

Contract BE/FE từng bị lệch nhưng đã được cập nhật lại trong tài liệu.
Kafka Producer/Consumer ban đầu cần thống nhất lại contract nhưng đã hoàn thành.
Frontend prototype dùng mock data nhưng đã chuyển sang API thật và empty state.

Nguyên nhân:

- Team muốn xử lý nhanh để không block flow.
- Một số lỗi được phát hiện trong lúc integration nên fix trực tiếp ngay.
- Chưa có thói quen tạo issue hoặc ghi MR note cho mọi thay đổi quan trọng.

Ảnh hưởng:

- Lịch sử thay đổi khó trace.
- Mentor hoặc member mới khó biết vấn đề từng xảy ra và đã được xử lý thế nào.
- Người sau có thể lặp lại lỗi vì không có ghi chú chính thức.
- Team khó phân biệt việc nào đã fix, việc nào còn open.

Action phòng tránh cho Sprint 2:

Từ Sprint 2 trở đi, nếu phát hiện bug, contract mismatch hoặc security concern, team cần tạo issue hoặc ghi rõ trong MR note. Không nên fix âm thầm mà không để lại trace.

Quy tắc đề xuất:

Bug nhỏ → ghi rõ trong MR note
 Bug ảnh hưởng flow/API/security → tạo issue riêng
 Contract thay đổi → cập nhật docs trong cùng MR
 Security issue → tạo issue riêng với priority high

11.8. Tổng hợp action items rút ra từ Challenges / Blockers

Vấn đề	Trạng thái	Action chính	Owner đề xuất	Sprint xử lý
Contract BE/FE từng bị lệch	Đã xử lý	Freeze/update API Contract trước khi FE integrate	Nhân + service owner	Sprint 2 trở đi
Gateway expose internal API	Cần xử lý	Remove/protect <code>/internal/users/**</code>	Khang + Nhân	Sprint 2
Kafka mới với team	Đã xử lý flow chính	Event Contract bắt buộc trước khi code event mới	BE Lead + service owner	Sprint 2 trở đi
FE mock/prototype dễ gây hiểu nhầm	Đã xử lý Auth flow	Repo thật chỉ dùng API thật hoặc empty state	Nhân	Sprint 2 trở đi
Account not verified chưa đạt	Cần xử lý	Fix <code>AUTH_ACCOUNT_NOT_VERIFIED</code> response và FE redirect	Vinh + Nhân	Sprint 2
Docs chưa sync code ở mọi thời điểm	Đã cải thiện, cần chuẩn hóa	MR phải có checklist cập nhật tài liệu	Cả team	Sprint 2 trở đi
Fix nhanh nhưng chưa có issue	Cần cải thiện quy trình	Bug/contract/security fix phải có issue hoặc MR note	Cả team	Sprint 2 trở đi

11.9. Working Agreement cho Sprint 2

Để tránh lặp lại các lỗi Sprint 1, team thống nhất các nguyên tắc sau cho Sprint 2:

- Mỗi API mới phải có API Contract trước hoặc song song với implementation.
- Mỗi event Kafka mới phải có Event Contract trước khi code Producer/Consumer.
- Internal API mặc định không expose qua API Gateway.
- Frontend repo thật không dùng mock business data.
- Mỗi MR phải kiểm tra tài liệu liên quan đã được cập nhật.

- Mỗi bug/contract/security fix phải có issue hoặc MR note.
 - Mỗi issue bug/blocker phải có action item và owner xử lý.
 - Mỗi service nên có một owner chính để tránh conflict code.
 - Các lỗi đã ghi trong retrospective phải được tạo issue nếu cần xử lý tiếp.
-

12. Team Member Contributions

Nhân

- Frontend Auth Integration.
- Kéo/migrate các màn từ FE prototype về repo thật.
- Register/Login/Verify OTP UI.
- CCCD Check integration.
- Token storage.
- Frontend integration documentation.
- E2E auth flow testing.
- BE lead / integration review cho Auth - User - Kafka flow.

Vinh

- Auth Service.
- Register/Login/Verify/Refresh Token.
- JWT/Refresh Token.
- Kafka Producer publish **ACCOUNT_CREATED**.
- Auth API contract updates.

Thành

- User Service.
- Internal Create User Profile API.
- Get User Profile API.
- Kafka Consumer consume **ACCOUNT_CREATED**.
- Create user profile from event.
- User API contract updates.

Khang

- API Gateway route support.
- Gateway routing for Auth/User services.
- Support infrastructure/config review.

Hoàng

- Support planning/documentation/backend module preparation.
 - Candidate for Notification/Showtime-related tasks in Sprint 2 depending on workload.
-

13. Action Items Before Closing Sprint 1

- ☐ Fix hoặc tạo bug issue cho Gateway expose `/internal/users/**`.
- ☐ Fix hoặc tạo issue cho `AUTH_ACCOUNT_NOT_VERIFIED` handling.
- ☐ Đồng bộ Login response field `token` hoặc update contract.
- ☐ Cập nhật `user-service-api.md` với Kafka Consumer status nếu chưa merge.
- ☐ Cập nhật `account-created-event.md` với implementation status/pass result.
- ☐ Merge các MR còn pending vào `develop`.
- ☐ Chuẩn bị demo script cho mentor.
- ☐ Tạo Sprint 2 milestone.
- ☐ Tạo Sprint 2 backlog.

14. Sprint 2 Direction

Sprint 2 sẽ không mở thêm service mới ngoài scope hiện tại của project. Scope dự án được gói gọn trong các service đã có trong thư mục `server/`:

```
analytics-service
auth-service
booking-service
movie-service
notification-service
payment-service
promotion-service
score-service
user-service
api-gateway
```

Trong Sprint 1, team đã hoàn thành nền tảng Authentication, User Profile, Frontend Auth Integration và Kafka `ACCOUNT_CREATED` flow. Ngoài ra, các schema cho các service chính cũng đã được tạo thành công. Vì vậy, Sprint 2 **không tập trung vào tạo schema mới**, mà tập trung vào việc implement API foundation dựa trên schema đã có.

Định hướng Sprint 2 là mỗi thành viên phụ trách một service riêng để tránh xung đột code khi nhiều người cùng sửa một service. Cách chia này giúp team dễ quản lý branch, Merge Request và review hơn.

Định hướng Sprint 2:

```
Implement API foundations for the remaining existing services based on completed
schemas, with each member owning one service to reduce merge conflicts and improve
development ownership.
```

Các nghiệp vụ như phim, suất chiếu, ghế, đặt vé, thanh toán, khuyến mãi, điểm thưởng, thông báo và thống kê sẽ được xử lý trong các service hiện có, không tách thêm service mới.

15. Sprint 2 Goal

Sprint 2 Goal đề xuất:

Implement API foundations for the remaining LoraFilm services based on completed database schemas, while assigning each developer ownership of one service to reduce code conflicts and prepare for integration, performance optimization and deployment in later sprints.

Mục tiêu cụ thể:

- Mỗi service còn lại có API Contract rõ ràng.
- Mỗi service có các API foundation chạy được ở local.
- Các API public/protected đi qua API Gateway.
- Các internal API nếu có phải được ghi rõ và không expose sai qua Gateway.
- Frontend có thể bắt đầu nối các màn hình tương ứng bằng API thật hoặc empty state rõ ràng.
- Các sprint sau có thêm thời gian cho integration, performance, testing và deploy.

16. Sprint 2 Backlog Draft

16.1. Movie Service

Owner đề xuất: Thành **Service:** `movie-service`

Movie Service phụ trách dữ liệu phim, thể loại phim, thông tin hiển thị phim và các dữ liệu liên quan đến suất chiếu nếu schema hiện tại đang đặt trong service này.

Schema hiện tại đã có các nhóm bảng như:

```
movies
genres
movies_genres
rooms
seats
showtimes
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của `movie-service`
- ☐ Viết hoặc cập nhật `docs/api/movie-service-api.md`
- ☐ Implement API lấy danh sách phim
- ☐ Implement API lấy chi tiết phim
- ☐ Implement API quản lý phim cơ bản cho admin nếu kịp
- ☐ Implement API lấy danh sách thể loại phim
- ☐ Implement API lấy suất chiếu theo phim nếu nằm trong scope service
- ☐ Implement API lấy phòng/ghế/suất chiếu nếu schema hiện tại đang thuộc movie-service
- ☐ Cấu hình route Gateway cho `/api/movies/**`
- ☐ Test API bằng curl/Postman/Swagger

- ☐ Seed data tối thiểu để FE có dữ liệu thật

Expected documents:

```
docs/api/movie-service-api.md
```

Expected routes:

```
GET /api/movies
GET /api/movies/{movieId}
GET /api/genres
GET /api/movies/{movieId}/showtimes
```

16.2. Booking Service

Owner đề xuất: Vinh **Service:** `booking-service`

Booking Service phụ trách đặt vé, vé, giữ ghế tạm thời và trạng thái booking.

Schema hiện tại đã có các nhóm bảng như:

```
bookings
tickets
seat_reservations
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của `booking-service`
- ☐ Viết hoặc cập nhật `docs/api/booking-service-api.md`
- ☐ Implement API tạo booking draft hoặc booking mới
- ☐ Implement API lấy chi tiết booking
- ☐ Implement API lấy danh sách booking theo user
- ☐ Implement API tạo ticket sau khi booking thành công nếu kịp
- ☐ Implement API giữ ghế tạm thời dựa trên `seat_reservations`
- ☐ Chuẩn bị hướng tích hợp Redis cho seat lock nếu chưa implement ngay
- ☐ Cấu hình route Gateway cho `/api/bookings/**`
- ☐ Test booking foundation bằng dữ liệu thật

Expected documents:

```
docs/api/booking-service-api.md
```

Expected routes:

```
POST /api/bookings
GET /api/bookings/{bookingId}
GET /api/bookings/user/{userId}
POST /api/bookings/seat-reservations
```

16.3. Payment Service

Owner đề xuất: Vinh **Service:** `payment-service`

Payment Service phụ trách thanh toán, trạng thái thanh toán và log thay đổi trạng thái thanh toán.

Schema hiện tại đã có các nhóm bảng như:

```
payments
payment_logs
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của `payment-service`
- ☐ Viết hoặc cập nhật `docs/api/payment-service-api.md`
- ☐ Implement API tạo payment cho booking
- ☐ Implement API mock confirm payment
- ☐ Implement API cập nhật trạng thái payment
- ☐ Implement API lấy payment theo booking
- ☐ Ghi log thay đổi trạng thái vào `payment_logs`
- ☐ Cấu hình route Gateway cho `/api/payments/**`
- ☐ Chuẩn bị hướng tích hợp VNPay/MoMo ở sprint sau nếu cần

Expected documents:

```
docs/api/payment-service-api.md
```

Expected routes:

```
POST /api/payments
GET /api/payments/booking/{bookingId}
POST /api/payments/{paymentId}/confirm
POST /api/payments/{paymentId}/cancel
```

16.4. Promotion Service

Owner đề xuất: Khang **Service:** `promotion-service`

Promotion Service phụ trách chiến dịch khuyến mãi, mã giảm giá và lịch sử sử dụng khuyến mãi.

Schema hiện tại đã có các nhóm bảng như:

```
promotion_campaigns
promotions
promotion_usages
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của **promotion-service**
- ☐ Viết hoặc cập nhật **docs/api/promotion-service-api.md**
- ☐ Implement API lấy danh sách promotion đang active
- ☐ Implement API validate promotion code
- ☐ Implement API tính discount cơ bản
- ☐ Implement API ghi nhận promotion usage nếu kịp
- ☐ Implement API quản lý campaign/promotion cơ bản cho admin nếu kịp
- ☐ Cấu hình route Gateway cho **/api/promotions/****
- ☐ Test các case expired, inactive, usage limit

Expected documents:

```
docs/api/promotion-service-api.md
```

Expected routes:

```
GET /api/promotions/active
POST /api/promotions/validate
POST /api/promotions/apply
```

16.5. Score Service

Owner đề xuất: Khang **Service:** **score-service**

Score Service phụ trách điểm thưởng, hạng thành viên và lịch sử thay đổi điểm.

Schema hiện tại đã có các nhóm bảng như:

```
membership_tiers
user_scores
score_history
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của `score-service`
- ☐ Viết hoặc cập nhật `docs/api/score-service-api.md`
- ☐ Implement API lấy điểm hiện tại của user
- ☐ Implement API lấy lịch sử điểm
- ☐ Implement API cộng điểm sau booking thành công nếu kịp
- ☐ Implement API trừ điểm/redeem điểm nếu kịp
- ☐ Implement logic xác định membership tier cơ bản
- ☐ Cấu hình route Gateway cho `/api/scores/**`
- ☐ Test score balance và score history

Expected documents:

```
docs/api/score-service-api.md
```

Expected routes:

```
GET /api/scores/users/{userId}
GET /api/scores/users/{userId}/history
POST /api/scores/users/{userId}/add
POST /api/scores/users/{userId}/redeem
```

16.6. Notification Service

Owner đề xuất: Hoàng **Service:** `notification-service`

Notification Service phụ trách template thông báo và log gửi thông báo.

Schema hiện tại đã có các nhóm bảng như:

```
notification_templates
notification_logs
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của `notification-service`
- ☐ Viết hoặc cập nhật `docs/api/notification-service-api.md`
- ☐ Implement API tạo notification log
- ☐ Implement API lấy notification logs theo user
- ☐ Implement API quản lý notification template cơ bản nếu kịp
- ☐ Chuẩn bị internal API cho các service khác gọi gửi thông báo
- ☐ Quyết định endpoint nào public, endpoint nào internal
- ☐ Cấu hình route Gateway nếu có API public
- ☐ Không bắt buộc gửi email thật trong Sprint 2

Expected documents:

```
docs/api/notification-service-api.md
```

Expected routes:

```
GET /api/notifications/users/{userId}
POST /internal/notifications
```

16.7. Analytics Service

Owner đề xuất: Khang **Service:** `analytics-service`

Analytics Service phụ trách thống kê doanh thu theo ngày và doanh thu theo phim.

Schema hiện tại đã có các nhóm bảng như:

```
daily_revenue_stats
movie_revenue_stats
```

Scope Sprint 2:

- ☐ Review lại schema hiện có của `analytics-service`
- ☐ Viết hoặc cập nhật `docs/api/analytics-service-api.md`
- ☐ Implement API lấy thống kê doanh thu theo ngày
- ☐ Implement API lấy thống kê doanh thu theo phim
- ☐ Implement API tổng quan dashboard admin nếu kịp
- ☐ Chuẩn bị hướng cập nhật stats từ booking/payment ở sprint sau
- ☐ Cấu hình route Gateway cho `/api/analytics/**`
- ☐ Test với seed data hoặc dữ liệu giả ở database, không hardcode ở frontend

Expected documents:

```
docs/api/analytics-service-api.md
```

Expected routes:

```
GET /api/analytics/revenue/daily
GET /api/analytics/revenue/movies
GET /api/analytics/dashboard
```

16.8. Auth Service Stabilization

Owner đề xuất: Vinh **Service:** `auth-service`

Auth Service đã hoàn thành nền tảng ở Sprint 1. Sprint 2 chỉ cần ổn định lại các điểm lệch contract hoặc issue bảo mật.

Scope Sprint 2:

- ☐ Đồng bộ Login response nếu còn lệch field `token`
 - ☐ Fix hoặc chuẩn hóa `AUTH_ACCOUNT_NOT_VERIFIED`
 - ☐ Đảm bảo response login có `accountId` hoặc JWT claim thống nhất
 - ☐ Review Refresh Token flow
 - ☐ Cập nhật tài liệu nếu implementation thay đổi
-

16.9. User Service Stabilization

Owner đề xuất: Thành **Service:** `user-service`

User Service đã hoàn thành nền tảng ở Sprint 1. Sprint 2 chỉ cần ổn định lại sau Kafka flow.

Scope Sprint 2:

- ☐ Đảm bảo Kafka Consumer vẫn ổn định
 - ☐ Đảm bảo public profile không trả full CCCD
 - ☐ Đồng bộ naming `verifiedPhone`
 - ☐ Giữ `/internal/users` là internal fallback/debug API
 - ☐ Cập nhật tài liệu nếu implementation thay đổi
-

16.10. API Gateway Stabilization

Owner đề xuất: Nhân **Service:** `api-gateway`

API Gateway cần được review kỹ trong Sprint 2 vì đã phát hiện route internal bị expose.

Scope Sprint 2:

- ☐ Remove hoặc protect route `/internal/users/**`
- ☐ Cấu hình route cho các service trong scope hiện tại
- ☐ Đảm bảo Frontend chỉ gọi `/api/**`
- ☐ Không expose internal API không cần thiết
- ☐ Review CORS config
- ☐ Test route bằng curl/Postman

Expected Gateway routes:

```
/api/auth/**  
/api/users/**  
/api/movies/**
```

```
/api/bookings/**
/api/payments/**
/api/promotions/**
/api/scores/**
/api/notifications/** nếu public
/api/analytics/** nếu public/protected
```

17. Suggested Sprint 2 Task Distribution

Nguyên tắc phân công Sprint 2:

Mỗi người ôm một service chính để giảm conflict code.
 Một service chỉ nên có một owner chính.
 Các member khác chỉ support review, test hoặc contract nếu cần.

Member	Service chính	Trách nhiệm chính
Nhân	Frontend + Integration Review / Analytics nếu cần	Nối FE khi API sẵn sàng, review contract, hỗ trợ test E2E
Vinh	Booking Service và Payment Service	Implement logic và API foundation cho luồng đặt vé (Booking), quản lý ghế (Seat Reservation) và vé (Ticket). Tích hợp và xử lý luồng thanh toán (Payment).
Thành	Movie Service	Phát triển API foundation cho việc quản lý thông tin phim, suất chiếu (Movie/Showtime).
Khang	Analytics Service, Score Service	Hỗ trợ nền tảng cho dịch vụ thống kê (Analytics). Phát triển hệ thống tính điểm thưởng/đánh giá (Score).
Hoàng	Notification Service, Promotion Service	Thiết lập hệ thống gửi thông báo (Notification). Implement API foundation cho hệ thống khuyến mãi, mã giảm giá (Promotion).

Ghi chú:

- Nếu workload quá lớn, có thể giảm bớt scope từng service xuống mức API foundation.
- Không nên để 2 người cùng code một service trong cùng thời điểm nếu không cần thiết.
- API contract nên được tạo trước hoặc song song với implementation.
- Các MR vẫn target vào **develop**.

18. Sprint 2 Documentation Requirements

Vì schema đã được tạo thành công, Sprint 2 không yêu cầu tạo schema từ đầu. Tuy nhiên, nếu implementation cần chỉnh field nhỏ thì phải cập nhật lại schema file tương ứng.

Mỗi service/module trong Sprint 2 cần có tối thiểu:

```
docs/api/<service-name>-api.md
```

Nếu có thay đổi database:

```
docs/database/mysql/<service-name>-schema.sql
```

Nếu có event:

```
docs/events/<event-name>.md
```

Mỗi API contract cần có:

- Endpoint.
- Method.
- Request body.
- Response success.
- Response error.
- Status code.
- Business rules.
- Gateway URL.
- Direct service URL.
- Auth/security notes.
- Scope chưa bao gồm.

19. Sprint 2 Definition of Done

Sprint 2 được xem là hoàn thành khi:

- ☐ Mỗi service được giao có owner rõ ràng.
- ☐ Không phát sinh service mới ngoài scope đã chốt.
- ☐ API contract cho service được giao được tạo hoặc cập nhật.
- ☐ API foundation của service chạy được ở local.
- ☐ API Gateway route tương ứng được cấu hình nếu service có public API.
- ☐ Internal API không bị expose sai qua Gateway.
- ☐ Frontend có thể bắt đầu gọi API thật hoặc hiển thị empty state rõ ràng.
- ☐ Các MR được review và merge vào **develop**.
- ☐ Các limitation được ghi rõ để xử lý ở sprint sau.

20. Sprint 2 Risks

Các rủi ro khi mỗi người ôm một service trong Sprint 2:

- Scope vẫn lớn dù đã chia theo service.
- Một số service phụ thuộc nhau như Booking, Payment, Score, Promotion.
- Gateway route nhiều, dễ cấu hình sai.
- API response giữa các service dễ lệch format nếu không có contract.
- Frontend có thể bị block nếu backend chưa có seed data.
- Nếu một member bị kẹt service của mình, cả flow tích hợp sau đó có thể bị chậm.

Cách giảm rủi ro:

- Mỗi service cần issue rõ scope.
- Ưu tiên API foundation trước, chưa cần hoàn thiện full logic phức tạp.
- Tạo seed data tối thiểu.
- Review API contract sớm.
- Test service riêng trước khi test flow liên service.
- Ghi rõ dependency giữa các service.
- Không sửa service của người khác nếu không trao đổi trước.